UNIVERSITAS MULTI DATA PALEMBANG
Fakultas Ilmu Komputer dan Rekayasa
Program Studi Sistem Informasi

TEMPLAT LAPORAN

TUGAS 2

Postman Collection dan Automated Test Script

Pengujian *RESTful API* dengan *Postman Automation*

Pengembangan Aplikasi Web II | SUB-CPMK-7

Nama dan NIM	Natalia Siddharta 2428240096
Kelas	SI5C
Judul project / topik API	RESTful API Akademik: Entitas Mata Kuliah (/courses)
Tautan repositori	
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	19 Oktober 2026

Petunjuk Pengisian

Dokumen ini adalah laporan **Tugas 2**: menguji RESTful API project akhir Anda dengan Postman. Seluruh isinya menggunakan **API milik Anda sendiri**, bukan API Mahasiswa pada contoh materi. Pelajari langkahnya pada materi Pertemuan 8 sebelum mengisi.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya `[sumber-daya]`, dengan data API Anda.
2. Bukti berupa gambar (*screenshot*) diisi pada kotak bergaris putus-putus. Cuplikan kode test script **ditempel sebagai teks**, bukan gambar.
3. Setiap bukti memiliki keterangan dan kotak **Penjelasan**. Tulis penjelasan dengan kalimat Anda sendiri dan sebut nama *request* atau fungsi dari API Anda.
4. Simpan hasil akhir sebagai PDF, lalu kumpulkan melalui LMS sesuai ketentuan dosen (format nama berkas dan batas waktu mengikuti petunjuk di LMS).

B. Bukti yang diminta

No	Bukti	Bagian	Jenis
1	Daftar request dalam Collection	Bab 2	Screenshot (wajib)
2	Cuplikan test script	Bab 3	Kode teks (wajib)
3	Ringkasan Collection Runner	Bab 4	Screenshot (wajib)
4	Hasil Newman di terminal	Bab 5	Screenshot (pengayaan)

C. Syarat tangkapan layar

Bukti	Yang harus terlihat
Collection	Nama Collection, kedua folder (bila ada), dan seluruh nama request. Buka semua folder sebelum mengambil gambar.
Ringkasan Runner	Nama Collection, Environment yang dipakai, jumlah request, jumlah test **lulus** dan **gagal**. Seluruh test harus lulus (0 gagal).
Newman	Perintah yang diketik, daftar hasil, dan tabel `executed` dan `failed` di bagian akhir.
Larangan	Gambar dipotong sehingga angka hasil tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

> **Sebelum mengambil screenshot Runner**
> Nyalakan ulang server agar data kembali ke kondisi awal, pilih Environment yang benar, lalu jalankan seluruh Collection. Bila ada test gagal, perbaiki dulu dan jalankan ulang sampai 0 gagal.

Bab 1 Identitas API dan Daftar Endpoint

Bab ini menjelaskan API yang diuji, agar pembaca memahami cakupan pengujian pada bab berikutnya.

1.1 Deskripsi singkat API

API yang dibangun merupakan RESTful API murni yang mengelola entitas Mata Kuliah (/courses) pada sistem informasi akademik. Backend ini dibangun menggunakan runtime Node.js dan framework Express.js, di mana penyimpanan data bersifat in-memory (array di RAM) dengan mekanisme manipulasi data CRUD lengkap serta fitur filter *query string* tanpa antarmuka visual (murni mengembalikan format data JSON).

1.2 Daftar endpoint

Tuliskan **seluruh** *endpoint* API Anda. Kolom "API key" diisi Ya bila rute dilindungi `cekApiKey` atau mekanisme serupa.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/courses	Tidak	200	Mengambil seluruh daftar data mata kuliah
2	GET	/courses/:id	Tidak	200	Mengambil satu data mata kuliah spesifik berdasarkan ID
3	GET	/courses?semester={nilai}	Tidak	200	Menyaring (filter) daftar mata kuliah berdasarkan semester
4	POST	/courses	Tidak	201	Menambahkan entitas data mata kuliah baru ke sistem
5	PUT	/courses/:id	Tidak	200	Memperbarui seluruh atribut data mata kuliah secara penuh
6	DELETE	/courses/:id	Tidak	200	Menghapus data mata kuliah tertentu dari sistem

1.3 Variabel Environment

Tuliskan variabel yang Anda simpan di Environment. Jangan menuliskan nilai rahasia sungguhan.

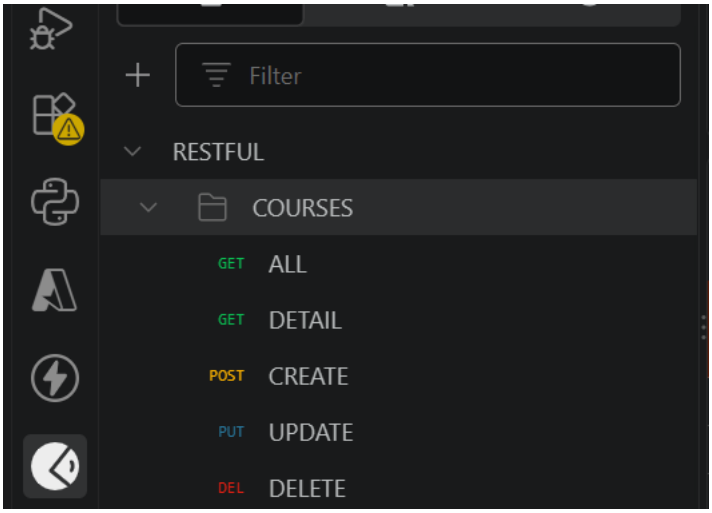
No	Nama variabel	Nilai (contoh)	Fungsi
1	base_url	http://localhost:4000	Alamat basis host dan port server backend lokal Express.js
2	created_course_id	4 *(opsional diisi via script)*	Menyimpan ID mata kuliah baru hasil pembuatan dari request POST

Bab 2 Postman Collection

Bab ini membuktikan bahwa seluruh *endpoint* sudah dikelompokkan dalam satu Collection.

2.1 Daftar request dalam Collection

Harus terlihat pada screenshot: nama Collection dan seluruh request di dalam folder. Buka semua folder.



Gambar 1. Collection beserta seluruh request

Penjelasan:
Request dikelompokkan ke dalam satu collection bernama RESTFUL dengan subfolder COURSES. Urutan request disusun secara terstruktur mengikuti siklus hidup data: diawali dengan pembacaan data awal (ALL, DETAIL, FILTER), dilanjutkan dengan penambahan data baru (CREATE), pembaruan record (UPDATE), dan diakhiri dengan penghapusan (DELETE). Urutan ini dirancang agar data awal tidak terhapus sebelum sempat diuji oleh request ALL dan DETAIL.

2.2 Tabel request

Salin daftar request pada gambar di atas ke tabel berikut. Setiap *endpoint* pada Bab 1 harus punya minimal satu request.

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
1	ALL	GET	{{base_url}}/courses	COURSES
2	DETAIL	GET	{{base_url}}/courses/1	COURSES
3	FILTER	GET	{{base_url}}/courses?semester=3	COURSES
4	CREATE	POST	{{base_url}}/courses	COURSES
5	UPDATE	PUT	{{base_url}}/courses/1	COURSES
6	DELETE	DELETE	{{base_url}}/courses/1	COURSES

Bab 3 Automated Test Script

Bab ini menunjukkan *assertion* yang Anda tulis. Setiap request wajib memiliki minimal satu *assertion* status code dan satu *assertion* struktur atau isi data JSON.

3.1 Rekap assertion per request

No	Nama request	Assertion status code	Assertion struktur atau isi JSON
1	ALL	pm.response.to.have.status(200)	pm.expect(pm.response.json()).to.be.an("array")
2	DETAIL	pm.response.to.have.status(200)	pm.expect(pm.response.json().id).to.eql(1)
3	FILTER	pm.response.to.have.status(200)	Memastikan seluruh item array bernilai semester === 3
4	CREATE	pm.response.to.have.status(201)	pm.expect(pm.response.json().status).to.eql("success")
5	UPDATE	pm.response.to.have.status(200)	pm.expect(pm.response.json().status).to.eql("success")
6	DELETE	pm.response.to.have.status(200)	pm.expect(pm.response.json().data).to.be.null

3.2 Cuplikan 1: request GET

Tempel test script **satu request GET** pilihan Anda.

```
// Test script request GET: ALL
pm.test("Status 200 OK", function () {
    pm.response.to.have.status(200);
});

pm.test("Data berupa array", function () {
    pm.expect(pm.response.json()).to.be.an("array");
});
```

Penjelasan:

Pengujian pertama (Status 200 OK) memastikan bahwa server berhasil merespons permintaan baca klien tanpa adanya galat server. Pengujian kedua (Data berupa array) memvalidasi arsitektur respons RESTful murni di mana rute GET all harus langsung mengembalikan array daftar objek data tanpa pembungkus metadata berlebih.

3.3 Cuplikan 2: request POST atau request yang menyimpan variabel

Tempel test script **request POST** (atau request lain yang menyimpan variabel ke Environment).

```
// Test script request POST: CREATE
pm.test("Status 201 Created", function () {
    pm.response.to.have.status(201);
});

pm.test("Status success", function () {
    const res = pm.response.json();
    pm.expect(res.status).to.eql("success");
    pm.expect(res.data).to.be.an("object");
});
```

Penjelasan:

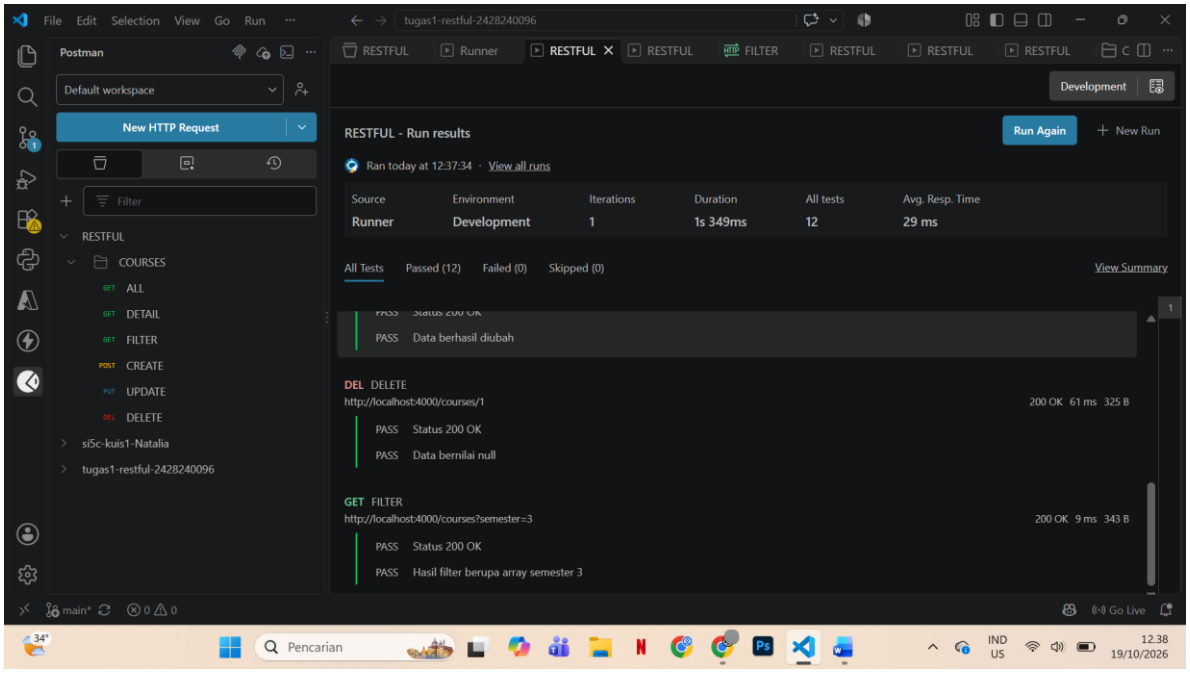
Pada request ini, skrip memvalidasi bahwa entitas mata kuliah baru berhasil diproses dan disimpan oleh server dengan status HTTP standar 201 Created. Respons JSON diverifikasi memiliki atribut status: "success" dan mengembalikan objek data yang memuat record mata kuliah baru beserta auto-increment ID yang dibuat oleh backend.

Bab 4 Hasil Collection Runner

Bab ini adalah bukti utama bahwa seluruh test lolos saat Collection dijalankan sekaligus.

4.1 Ringkasan Collection Runner

Harus terlihat pada screenshot: nama Collection, Environment, jumlah request, jumlah test lulus dan gagal. Seluruh test harus lulus.



Gambar 2. Ringkasan hasil Collection Runner

Penjelasan:

*Pengujian automated test dijalankan melalui Postman Collection Runner menggunakan environment **Development** (URL lokal port 4000). Total 6 request dieksekusi secara berurutan dan menghasilkan 12 assertions pengujian yang seluruhnya lulus (**100% Passed / 0 Failed**). Sebelum eksekusi dilakukan, server Express lokal telah di-restart terlebih dahulu agar kondisi data memori array kembali ke 3 entitas awal.*

4.2 Rekap angka hasil

Nama Collection	RESTUFL
Environment yang dipakai	Development
Jumlah request dijalankan	6
Jumlah test lulus	12
Jumlah test gagal	0
Tanggal dan jam pengujian	19 Oktober 2026, 12:37:34 WIB

4.3 Temuan selama pengujian

Catat kegagalan yang pernah muncul sebelum akhirnya lulus. Bagian ini menunjukkan bahwa pengujian benar-benar membantu menemukan masalah. Bila tidak ada, tulis "tidak ada kegagalan".

No	Request	Kegagalan (pesan)	Penyebab	Perbaikan
1	ALL	Empty request URL / Request could not be sent	Variabel base_url berstatus *Unresolved Variable* karena environment Development belum dipilih di pojok kanan atas, serta terjadi duplikasi protokol http://{{base_url}}.	Memilih environment Development aktif, memastikan variabel bernilai http://localhost:4000, serta memperbaiki URL menjadi murni {{base_url}}/courses.
2	ALL & DETAIL	TypeError: Cannot read properties of undefined (reading 'to')	Kode assertion pengujian ditempatkan pada tab *Pre-request Script* sehingga objek respons server belum terbentuk saat dievaluasi.	Memindahkan seluruh kode assertion ke dalam tab *Post-response (Tests)* agar dieksekusi sesaat setelah server mengirimkan respons.
3	UPDATE	400 Bad Request	Request body belum menyertakan JSON lengkap yang memenuhi seluruh field wajib model validasi backend.	Melengkapi raw JSON body dengan atribut wajib kode, namaMatkul, sks, dan semester bertipe data sesuai.
4	DELETE	404 Not Found	Data target ID 1 telah terhapus pada siklus pengujian runner sebelumnya.	Melakukan restart server lokal melalui terminal (npm run dev) agar array in-memory ter-reset ke kondisi awal sebelum runner dijalankan.

Bab 5 Newman di Terminal (Pengayaan)

> **Bagian ini opsional**
> Bab ini adalah pengayaan dan tidak mengubah kriteria penilaian. Isi bila Anda menjalankan Collection dengan Newman. Bila tidak, tulis "tidak dikerjakan" di bawah judul ini.

5.1 Hasil Newman

Harus terlihat pada screenshot: perintah `newman run ...`, daftar hasil, dan tabel `executed` dan `failed`.

TIDAK DIKERJAKAN

Gambar 3. Hasil menjalankan Collection dengan Newman

Penjelasan:

Tulis perintah yang Anda jalankan dan jelaskan arti exit code (0 atau 1) pada hasil Anda.

Bab 6 Repositori dan Berkas

Tautan repositori	https://github.com/nataliasiddharta/tugas1-restful-2428240096
Lokasi berkas Collection (.json)	Terletak pada root direktori repositori dengan nama file RESTFUL.postman_collection.json
Lokasi berkas Environment (.json)	Terletak pada root direktori repositori dengan nama file Development.postman_environment.json
Berkas Environment tidak memuat kunci rahasia sungguhan	[X] Ya [ ] Tidak
Repositori dapat dibuka dosen	[X] Ya [ ] Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan pengalaman Anda mengerjakan tugas ini.

Pertanyaan 1

Dari seluruh *assertion* yang Anda tulis, mana yang paling berguna untuk menemukan kesalahan pada API Anda? Berikan alasannya.

Penjelasan:

Assertion status code (pm.response.to.have.status(...)) adalah yang paling berguna karena langsung mendeteksi ketidaksesuaian kontrak HTTP secara instan. Ketika terjadi kesalahan validasi input (seperti field wajib yang kosong) atau rute yang salah penamaan, assertion status code langsung mendeteksi respons 400 Bad Request atau 404 Not Found sebelum terjadi kesalahan penguraian struktur data di tingkat aplikasi.

Pertanyaan 2

Apa perbedaan yang Anda rasakan antara menguji API secara manual dan memakai Collection Runner? Bagian mana yang paling sulit saat menulis test script, dan bagaimana Anda mengatasinya?

Penjelasan:

Menguji secara manual membutuhkan proses klik dan pengecekan respons satu per satu yang rentan terlewat, sedangkan Collection Runner mengeksekusi puluhan assertion secara otomatis dalam hitungan milidetik sehingga jauh lebih efisien untuk regresi pengujian. Bagian paling menantang adalah memahami urutan eksekusi siklus hidup Postman (antara Pre-request dan Post-response) serta memvalidasi struktur nested JSON, yang berhasil diatasi dengan memindahkan script pengujian ke tab Post-response dan mempelajari dokumentasi assertion Chai.js.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda `[...]` sudah diganti dengan data API saya	Sudah
2	Setiap endpoint API saya ada di Collection	Sudah
3	Alamat memakai `{{base_url}}`, bukan ditulis langsung	Sudah
4	Setiap request punya assertion status code dan struktur JSON	Sudah
5	Nama setiap test berupa kalimat yang menyatakan harapan	Sudah
6	POST menyimpan ID dengan pemeriksaan status, dan request berikutnya memakainya	Sudah
7	Server dinyalakan ulang sebelum Runner dijalankan	Sudah
8	Screenshot Runner menampilkan 0 test gagal dan angkanya terbaca jelas	Sudah
9	Berkas Environment tidak memuat kunci rahasia sungguhan	Sudah
10	Berkas disimpan sebagai PDF dan dikumpulkan sesuai ketentuan LMS	Sudah

Lampiran B Kriteria Penilaian

Kriteria berikut sesuai dengan slide materi Pertemuan 8. Bobot nilai mengikuti ketentuan dosen di LMS.

Kriteria	Deskripsi	Dinilai dari
Kelengkapan Postman Collection	Seluruh endpoint project akhir tercakup dalam satu Collection	Bab 1 dan Bab 2
Ketepatan automated test script	Assertion status code dan struktur response JSON tepat sasaran	Bab 3
Keberhasilan Collection Runner	Seluruh test lolos saat Collection dijalankan	Bab 4
Kejelasan dokumentasi hasil	Tangkapan layar ringkasan pengujian disertakan dengan jelas	Bab 4 dan seluruh dokumen